

My Codex Screenwriting Workflow

A Practical Method for Story Setup, Version Control, and Human-AI Collaboration

I wrote this workflow mainly for my future self.

If one day I get older and forget how to use Codex to write screenplays, at least I will have a working method I can return to. Technology will keep changing. ChatGPT, Codex, GitHub, and cloud drives may all change their interfaces, names, and rules, but the creative workflow itself can keep being revised and reused.

This is not the only correct method, and it is not meant to force anyone to follow a rigid checklist. It is simply the approach that, after testing, currently gives me the best balance between speed, quality, version control, and creative safety.

Overall, I divide the work into four layers:

1. ChatGPT handles early development: thinking through the story, organizing settings, and building structure.
2. Codex handles text execution: writing scenes, revising drafts, and organizing files according to specifications.
3. Other AI models handle critique: finding problems from different angles and proposing revision plans.
4. GitHub, Dropbox, and offline backups handle preservation: making every change traceable.

The human remains the final screenwriter, director, and producer. AI can help think, write, and critique, but what stays, what gets cut, and what the story ultimately says are still human decisions.

Part One: Initial Setup Workflow

Step 1: Create the Local Project Folder

First decide the project title. For example:

Feature screenplay: 0000

Then create the main project folder inside Dropbox or your main working drive:

```
0000/
```

I recommend separating the "workspace" from the "archive" instead of mixing everything together. The workspace is where daily changes happen. The archive is where you manually copy versions that, in principle, should no longer be automatically overwritten.

You can use a structure like this:

```
0000/
```

```
|— 01_Workspace/  
|— 02_Research/  
|— 03_Archive/  
|   |— Canon/  
|   |— Codex/  
|   |— Conversation/  
|— 04_Exports/
```

Dropbox can serve as the everyday synced workspace, but do not treat it as your only backup. Sync is convenient, but it also means accidental deletion, overwriting, or formatting damage may sync everywhere.

For truly important versions, regularly copy them by hand to another drive, a NAS, or an offsite folder that does not automatically sync.

Manual backup is annoying, but its value is that you must personally confirm: "This version is worth preserving."

Step 2: Create a Private GitHub Repository

Next, create a new repository on GitHub.

The naming convention does not need to be rigid, but I recommend using the English title plus the project type:

```
0000-Screenplay  
0000-Teleplay  
0000-Stageplay
```

For a feature film, use Screenplay. For a TV drama or series, use Teleplay. For a stage play, use Stageplay. The point is not to make the name elegant. The point is that, months later, you can immediately tell which work the repository belongs to.

The repository description can be simple:

```
Feature screenplay project: 0000
```

The most important rule is this: screenplay repositories should be **Private**.

Public repositories are useful for workflow templates, tools, blank templates, or teaching materials. Real screenplays that are still in development, not yet submitted, and not yet published should be kept in separate private repositories. Do not mix them with public templates.

GitHub's role here is not just "file storage." It preserves the differences, timestamps, and version history of every change. If later you need to find which version changed a character setting, which scene removed a line, or when a line of dialogue disappeared, Git history is much more reliable than a pile of files named "final", "final2", and "real-final".

Step 3: Align the Local Workspace with GitHub

First clarify the basic concept:

Codex mainly works inside your local project folder or worktree. GitHub is the remote repository.

So the workflow is not simply "give Codex the GitHub URL and call it done." The actual workflow is:

5. Create or clone the project folder locally.
6. Confirm that the folder is connected to the GitHub repository.
7. Open this local project folder in Codex.
8. After each work session, inspect the changes, commit, and push to GitHub.

I also recommend naming the Codex chat consistently:

```
Feature screenplay: 0000
```

When multiple projects are open at the same time, this prevents confusion.

At the start of each day, confirm which project, which version, which Canon, and which scene you are working on. This kind of small confirmation prevents AI from spending half a day working hard inside the wrong story.

Step 4: Do Story Research Before Asking AI to Improvise

For an entirely original project, you can first discuss the following with ChatGPT:

- Theme
- Genre
- Target audience
- Logline
- Story outline
- Character design
- Worldbuilding
- Structural beats
- Commercial positioning
- Budget scale
- Visual style

If the work is based on a true story, historical event, biography, case, profession, or specialized field, do research first.

AI is very good at quickly building a "map of questions." It can tell you which eras, professional rules, event sequences, locations, institutions, or legal contexts you need to investigate.

But do not treat AI as your only database.

This is especially important for stories involving real people, historical events, aviation, medicine, police or military work, law, finance, politics, religion, crime, or war. For these topics, create a Research Log:

```
Source title:
```

Source:
Date:
Key content:
Useful for which scene:
Items still requiring verification:

AI can help organize material, identify questions, and propose hypotheses, but key facts still need to be verified against reliable sources. Otherwise, you may end up with a screenplay that reads smoothly, sounds real, and is completely wrong.

Part Two: From Story Idea to Canon

Step 5: Build the Story from Small to Large

The basic order of screenwriting is not that different from traditional screenwriting theory.

Start by deciding:

9. Theme: What is this story really about?
10. Logline: Who is the protagonist, what is the goal, what is the obstacle, and what is the cost?
11. Genre: Romance, disaster, action, thriller, family drama, historical, science fiction, dark comedy?
12. Length: Short film, feature film, TV movie, series, stage play, or musical?
13. Structure: Three-act structure, Save the Cat, five-act structure, episodic beats, or another form?
14. Characters: Who is the protagonist, who is the opponent, who drives the story, and who carries the emotion?
15. Story outline: How do events unfold, where are the turns, and how does the ending land?

You do not need to force yourself to write five thousand words at the beginning.

Start with a one-thousand-word outline and check whether the direction is right. Then expand it gradually into a three-thousand-to-five-thousand-word synopsis. Character biographies, secrets, childhood experiences, relationships, and arcs can develop alongside the outline.

This stage is best for repeated discussion.

You can ask ChatGPT, ask other AI models, or deliberately ask different models to challenge your setup. A story does not become final in the first version. It gradually grows bones through repeated testing.

Step 6: Build the Canon and Narrative Spec

I usually call the screenplay settings document the Canon.

Canon is not a sacred bible that can never change. It is the current set of story rules that should not be casually violated.

A good Canon should include at least:

- Core theme

- World and time-space background
- Character ages, occupations, histories, and relationships
- Character motivations, secrets, weaknesses, and bottom lines
- Story timeline
- Major events that have already happened and cannot be overturned
- Non-negotiable settings
- Tone, style, and taboos
- Proper nouns, locations, organization names, and professional terminology
- Undecided questions that may still change

But Canon is more like a settings database.

When handing work to Codex, you also need a "current task specification." I call this the Narrative Spec.

The Narrative Spec must be more specific:

```
Which episode and which scene to write.
The dramatic purpose of this scene.
What each character wants.
How the conflict escalates.
What the audience should know after the scene ends.
Which established settings must not be changed.
Word count, page count, screenplay format, and tone requirements.
```

In short:

```
Canon is the world rulebook.
Narrative Spec is the construction blueprint for the current task.
```

Do not simply ask Codex to "write a scene according to the settings."

That instruction is too vague. It will easily fill in blanks on its own, add new settings, and decide character fates for you. You want an assistant, not a machine that takes over the writers' room when you are not looking.

Step 7: Build a 3C Archive System

I divide archive materials into three major categories, the 3C system:

```
Canon
Codex
Conversation
```

Canon

The Canon folder stores all story settings, character sheets, timelines, version specs, Narrative Specs, and research summaries.

The recommended master format is Markdown, meaning .md files.

Markdown has several advantages:

- Humans can read it directly.
- AI can understand it easily.
- It is not trapped inside Word formatting.
- It can be version tracked in Git.
- It can later be converted into Word, PDF, web pages, or other formats.

Use consistent file names:

```
0000_Canon_20260622_v001.md
0000_Narrative-Spec_20260622_v001.md
0000_Character-Bible_20260622_v002.md
```

Do not keep using names like "latest", "final version", or "final version actually final".

There will always be another version. Admit that first, and your files will be less likely to collapse into chaos.

Codex

The Codex folder stores each day's output.

I recommend creating a date folder for each workday:

```
20260622/
```

Inside it, you can store:

```
EP01_S01-S03_Draft.md
EP01_S01-S03_Draft.docx
EP01_S01-S03_Draft.pdf
Revision-Notes.md
Change-Log.md
```

One principle is important:

Markdown or plain-text screenplay files should be the master files. Word and PDF are output formats.

Word and PDF are good for reading, printing, submitting, and exchanging. They are not good for tracking fine-grained differences. For version management, Markdown, plain text, or another comparable text format is usually better.

Conversation

The Conversation folder preserves important conversations, work instructions, revision reasons, and decision summaries.

These can be .txt, .md, or exported conversation files.

Its purpose is not to guarantee that you can win any future copyright dispute. Its purpose is to preserve your creative trail:

- When you proposed an idea.
- How you revised a character.
- How you chose between versions.
- Which decisions were made by you as a human creator.

- Which ideas came from AI and were then revised or accepted by you.

In addition to conversation records, I recommend adding a simple Decision Log:

```
Date:  
Question:  
Options:  
Final decision:  
Reason:  
Affected scenes:
```

This is often more useful than saving a huge wall of chat history.

Part Three: Using Codex for Actual Writing

Step 8: Build the Project Work Structure

Once the story structure is reasonably mature, ask Codex to create the writing project.

You can use your own narrative-pipeline template and divide each project into:

```
00_brief  
01_development  
02_spec  
03_outline  
04_scenes  
05_critiques  
06_revisions  
07_exports
```

The advantage of this structure is that story settings, scene drafts, critique reports, revision versions, and final exports do not get mixed together.

Do not throw everything into one folder and expect your future self to remember which file is the real one.

You may remember now. Six months, three years, or ten years later, even you may not understand it.

Step 9: Build Style References, But Do Not Pretend AI Really Cloned You

Put your past screenplays, novels, short stories, stage plays, or dialogue scripts into a Style or references folder so Codex can use them as style references.

But I recommend not calling this "personality training" or "fully learning your style."

A more accurate description is:

You provide writing samples so the model can analyze and refer to your sentence patterns, rhythm, scene description, dialogue length, character interactions, narrative tendencies, and common tone.

Then ask it to organize a Style Guide:

Preferred sentence length:
Scene description density:
Dialogue rhythm:
Character speech habits:
Words to avoid:
Narrative problems to avoid:
Use of camera language:
Voice-over allowed or not:
Preference for silence or omission:

As long as the samples are your own work, or work you have permission to use, they can serve as reference material.

Other people can learn your process, and tools can be copied, but your aesthetic choices, life experience, emotional memory, value judgments, and way of seeing the world cannot be fully distilled just by dropping a few files into a folder.

AI can imitate surface features, but it does not know why you let a character remain silent for three seconds in a specific scene.

That choice is still yours.

Frankly, my own testing has also shown that Codex sometimes writes more concise, cleaner dialogue than I do, like a professional writer without emotional baggage. But when it tries to simulate my style, the result is usually still only partially convincing.

So style references are useful, but do not treat them as a technology for cloning a soul.

Step 10: Write the First Ten Pages as a Test

Do not ask Codex to write the entire screenplay at the start.

First write the first ten pages.

In Hollywood screenplay reading habits, the first ten pages are often the test that decides whether a reader keeps going. They do not need to explain everything, but they should at least make clear:

- What genre this is.
- Who the protagonist is.
- What problem the protagonist has.
- What is compelling about this world.
- Why the audience should keep watching.

If you plan to submit to a screenplay award, series award, film festival, grant, or competition, first feed Codex the format template, submission rules, page limits, font requirements, and taboos.

Do not wait until the script is finished to discover that the format is wrong and then spend a huge amount of time reformatting the whole thing.

For the first-ten-pages test, I recommend asking for three outputs at once:

1. Screenplay draft

2. Explanation of this scene's intent
3. List of issues that conflict with Canon or still require confirmation

This way, you are not only looking at the result. You can also see whether it actually understands your story.

Step 11: Ask Codex for a Writing Plan, Then Write in Batches

After the first ten pages are working, ask Codex to propose a writing plan.

Do not ask it to write the full screenplay in one go unless you are truly under a deadline and willing to accept major revision afterward.

Asking AI to write too much at once usually causes several problems:

- The later rhythm starts to loosen.
- Characters gradually lose their original voices.
- Foreshadowing from earlier sections is forgotten.
- Scenes become repetitive.
- There is a lot of content, but not much progress.
- Revision becomes more expensive than batch writing.

A more reasonable approach is to move ten to twenty pages at a time, or three to five scenes per work unit.

For a feature screenplay of around 120 pages, pure generation speed could theoretically produce a draft in six to twelve workdays. But once review, revision, version organization, and re-critique are included, two weeks is a more realistic estimate.

AI can be fast, but you do not have to lose control just because it can move quickly.

The real question is not "How many pages did I generate today?" The real question is "Are the pages I kept today still worth continuing tomorrow?"

Step 12: Use Multi-Model Critique, But Do Not Hand Creativity to a Vote

After each batch of draft pages is complete, you can send it to different AI models for critique.

For example:

- ChatGPT: story structure, character arcs, theme, and emotion.
- Claude: long-form reading, logical consistency, pacing, and character motivation.
- Gemini: information organization, research comparison, and scene problems.
- Grok or other models: alternative perspectives and blind spots.

But do not simply ask:

Is this story good?

That usually only produces vague praise.

Ask more specific questions:

```
Find the three weakest scenes in this episode.  
Identify places where character motivation is insufficient.  
Check whether this violates Canon.  
Analyze where the pacing starts to loosen.  
Find unnecessary expository dialogue.  
Propose three revision plans at different levels of intensity.  
Critique this separately from the perspective of an audience member, a producer, and  
a screenplay competition judge.
```

I call this "manual adversarial generation."

The point is not to make models fight each other. The point is to let different models find blind spots for you, and then you decide which comments are worth using.

It must always return to human judgment.

AI can give one hundred suggestions, but you do not need to accept them all. A screenplay is not a democracy, and the model with the highest score is not automatically right.

Step 13: Update Canon Before Revising the Script

New ideas will appear in the middle of writing. That is normal.

The problem is that many people immediately ask AI to rewrite the whole script as soon as they get a new idea. Afterward they discover that character settings, timeline, foreshadowing, and relationships have all been torn apart.

A better workflow is:

```
New idea  
-> Discuss feasibility  
-> Update Canon  
-> Update Narrative Spec  
-> Identify affected scenes  
-> Ask Codex for targeted revisions  
-> Produce Change Log  
-> Human review
```

This is not only about saving model usage. More importantly, it keeps the story system from falling apart.

ChatGPT is better for brainstorming, discussion, organization, and exploring possibilities. Codex is better for executing clear specifications, organizing files, rewriting scenes, and producing versions.

One is closer to a writers' room discussion. The other is closer to production execution.

Separating the work is usually more efficient than asking one tool to do everything.

Part Four: Daily Wrap-Up and Version Preservation

Step 14: Fixed End-of-Day Workflow

At the end of each workday, I recommend completing the following:

1. Read the day's output.
2. Confirm which content is officially accepted.
3. Update Canon or Narrative Spec.
4. Write Change Log and Decision Log.
5. Inspect file differences.
6. Commit the day's version.
7. Push to GitHub.
8. Manually copy important daily output to an offsite archive folder.
9. Save important conversation records.
10. Let Dropbox finish syncing only after the above is complete.

Do not write sloppy commit messages.

For example:

```
feat(ep01): complete first draft of S01-S03  
revise(canon): update Deng Ziqi's post-crash trauma setting  
fix(ep02): correct character age and timeline conflict  
critique: add Gemini and Claude critique summary
```

In the future, when you need to find "the day the father character changed", "the version that turned the ending into a dual-protagonist ending", or "the revision that deleted a scene", this will make the work much easier.

If you use automated writing, do not let automation directly modify the main working folder.

Ideally, automated tasks should output into an independent workspace or worktree. After you review, revise, and approve the result, merge it back into the official version.

Automation can help with repetitive work, but it should not bypass your review.

Part Five: After the First Draft, Return to Paper

Step 15: Print the First Draft

After the first draft is complete, export a PDF.

My own budget method is to use a home black-and-white laser printer, print thirty pages at a time double-sided, then use a two-hole punch and plastic binding strips.

It is not pretty, but it is cheap, sturdy, easy to flip through, and convenient for handwritten notes.

I do not particularly like brass fastener binding, especially when the script becomes thick. TV dramas or long series can easily reach more than five hundred pages. Brass fasteners loosen, pages scatter, and reading becomes awkward.

Of course, if you are already working professionally and need to hand material to a production company, agent, crew, or official jury, follow their required format.

But for creators still developing and repeatedly revising, readability matters more than ceremonial form.

I usually write the date and version by hand on the first page:

2026.06.22
Second draft

This is not just a ritual.

Reading on paper reveals problems that a screen hides: paragraphs that are too long, scenes that feel too similar, dialogue that is too dense, page-turn rhythm that does not work, and characters who all sound alike.

AI can speed up writing, but paper still forces you to slow down and look at the work.

Part Six: Publication, Submission, and Selling the Script

If an agent, producer, company, or collaborator is willing to continue developing the work, that is ideal.

If not, you can still consider:

- Entering screenplay awards, series awards, or related competitions.
- Publishing on a personal website, screenplay blog, or portfolio.
- Adapting it into a novel, comic, stage play, musical, or another IP form.
- Creating a proposal deck, character introduction, pitch deck, or sample reading package.
- Looking for filmmakers, directors, producers, or small studios to collaborate with.

In Taiwan, entering screenplay or series competitions is still one path for work to enter the industry's field of vision.

But competition rules always take priority over your personal workflow.

Word count, page count, anonymity rules, format, file name, cover page, public-disclosure restrictions, AI-collaboration rules, and required rights declarations all need to be checked one by one.

If the work is adapted from real people, news events, existing novels, historical material, or other people's works, you also need to assess sources, adaptation rights, privacy, reputation, and other possible risks.

Do not assume that because AI writes quickly, the work can skip these real-world issues.

Conclusion: Screenplays Are Made by Revision

The old line is still true:

Screenplays are made by revision.

Do not obsess over whether the first draft is a masterpiece. You do not need to force yourself to become Shakespeare in the first pass.

Before the work is filmed, seen by an audience, or truly enters the industry process, the most important thing is simple:

Finish it.

After finishing it, revise it.

After revising it, show it to other people.

Even if the work actually sells, it may later be changed drastically by directors, producers, actors, platforms, budgets, and production conditions.

That is not creative failure. That is the reality of the film and television industry.

The only world you fully control is the free world that exists while you are writing.

AI can help organize research, generate scenes, fix formatting, track versions, analyze problems, and sometimes even write cleaner dialogue than you.

But why that story exists, why the character must be wounded, and why the ending must leave that one line behind are things only you know.

The process can be copied.

The tools can be shared.

What truly cannot be copied is your decision about which stories deserve to live.